

Отчёт по лабораторной работе №2

Компьютерный практикум по статистическому анализу данных

Арбатова Варвара Петровна

В данном отчёте представлены результаты выполнения лабораторной работы по теме “Структуры данных”

Описаны использованные методы, полученные экспериментальные данные и их анализ.

Сделаны выводы о достижении поставленной цели.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Цель работы	6
1.3	Задачи	6
1.4	Объект и предмет исследования	7
1.5	Методы исследования	7
1.6	Схема выполнения работы	7
2	Теоретическая часть	9
3	Материалы и методы	10
4	Выполнение лабораторной работы	11
4.1	Примеры: кортежи, словари, множества, массивы	11
4.2	Задание №1	15
4.3	Задание №2. Операции над множествами элементов разных типов .	15
4.4	Задание №3	17
4.5	Задание №4. Массив squares	23
4.6	Задание №5. Пакет Primes	23
4.7	Задание №6. Вычисление выражений	24
5	Результаты	26
6	Заключение	28
	Список литературы	29
A	Приложение А. Полный листинг программы	30
B	Приложение Б. Исходный ноутбук	31

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	8
-----	---	---

Список таблиц

1 Введение

1.1 Актуальность

Изучение структур данных является фундаментальной частью подготовки специалиста в области информатики. Язык Julia предоставляет широкий набор встроенных структур данных (кортежи, словари, множества, массивы), эффективная работа с которыми необходима при решении прикладных задач анализа данных и научных вычислений.

1.2 Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

1.3 Задачи

1. Изучить кортежи, словари, множества и массивы в Julia.
2. Выполнить операции над множествами элементов разных типов.
3. Решить задания 3.1–3.14 (построение массивов и векторов).
4. Создать массив `squares` из квадратов чётных чисел от 1 до 100.
5. Подключить пакет `Primes` и выполнить задания с простыми числами.
6. Вычислить три выражения (6.1–6.3).

1.4 Объект и предмет исследования

Объект — структуры данных языка Julia. Предмет — операции над ними.

1.5 Методы исследования

Практическое программирование, вычислительный эксперимент.

1.6 Схема выполнения работы

На рис. 1.1 представлена общая последовательность действий.

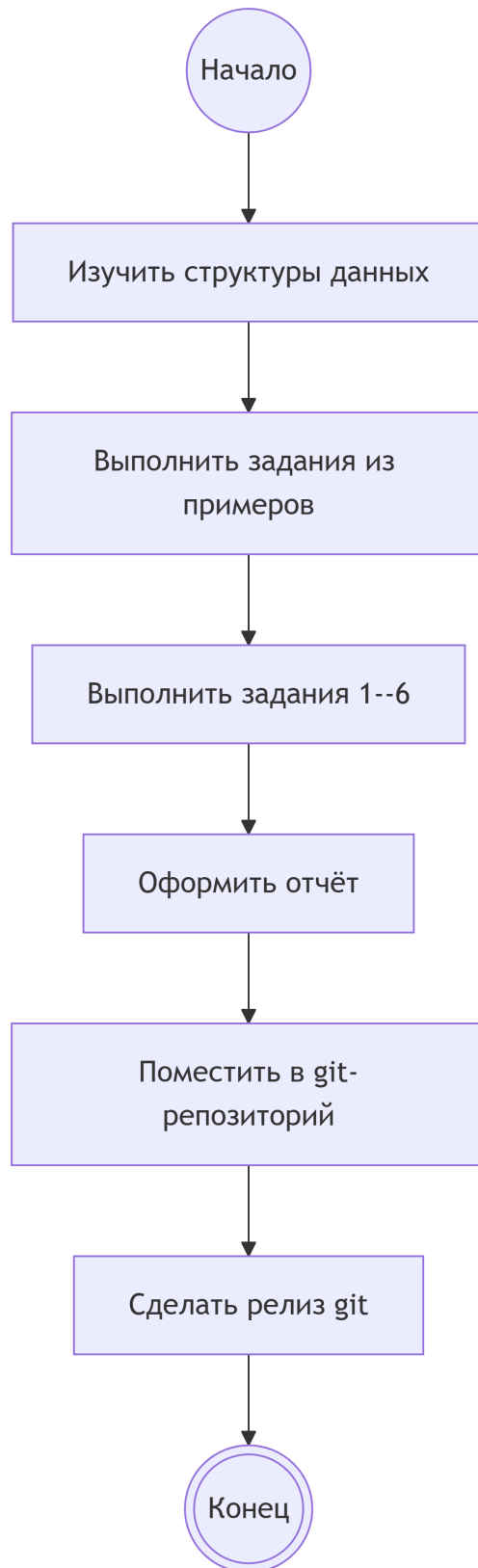


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

В Julia реализованы следующие структуры данных:

- **Кортежи (Tuple)** — неизменяемая индексируемая последовательность элементов.
- **Словари (Dict)** — неупорядоченный набор пар «ключ → значение».
- **Множества (Set)** — неупорядоченная совокупность уникальных элементов.
- **Массивы (Array)** — коллекция упорядоченных элементов в многомерной сетке.

Основные общие функции: `isempty()`, `length()`, `in()`, `unique()`, `reduce()`, `maximum()`/`minimum()`.

Подробнее о структурах данных в Julia см. в [1; 2; 4]. О пакете `Primes` см. в [3].

3 Материалы и методы

- **Язык:** Julia 1.12.
- **Среда:** Jupyter Notebook (IJulia).
- **Пакеты:** Random (входит в стандартную библиотеку), Primes v0.5.7.
- **Оборудование:** персональный компьютер.

4 Выполнение лабораторной работы

4.1 Примеры: кортежи, словари, множества, массивы

4.1.1 Кортежи

```
# empty tuple:  
()  
# tuple of strings:  
favoritelang = ("Python", "Julia", "R")  
# tuple of integers:  
x1 = (1, 2, 3)  
# tuple of mixed types:  
x2 = (1, 2.0, "tmp")  
# named tuple:  
x3 = (a=2, b=1+2)  
  
println(favoritelang, x1, x2, x3)
```

("Python", "Julia", "R")(1, 2, 3)(1, 2.0, "tmp")(a = 2, b = 3)

```

# length of tuple x2:
length(x2)
# access elements of tuple x2:
println(x2[1], x2[2], x2[3])
# add the second and third elements of tuple x1:
c = x1[2] + x1[3]
println(c)
# access elements of named tuple x3:
println(x3.a, x3.b, x3[2])
# check membership of "tmp" and 0 in x2:
in("tmp", x2), 0 in x2

```

12.0tmp

5

233

(true, false)

4.1.2 Словари

```

# create a dictionary named phonebook:
phonebook = Dict{"XXXXXXXX XX.X." => ("867-5309", "333-5544"),
                "XXXXXXXXXXXXXX" => "555-2368")
# print dictionary keys:
keys(phonebook)
# print dictionary values:
values(phonebook)
# print key-value pairs:
pairs(phonebook)
# check if a key exists:

```

```

haskey(phonebook, "XXXXXXXX X.X.")
# add a new entry:
phonebook["XXXXXXXX X.X."] = "555-3344"
# remove a key:
pop!(phonebook, "XXXXXXXX X.X.")
# merge dictionaries:
a = Dict("foo" => 0.0, "bar" => 42.0);
b = Dict("baz" => 17, "bar" => 13.0);
merge(a, b), merge(b, a)

```

4.1.3 Множества

```

# create a set of four integers:
A = Set([1, 3, 4, 5])
# create a set of characters from a string:
B = Set("abrakadabra")
# check equality of two sets:
S1 = Set([1,2]); S2 = Set([3,4]);
issetequal(S1, S2)
# check equality after deduplication:
S3 = Set([1,2,2,3,1,2,3,2,1]); S4 = Set([2,3,1]);
issetequal(S3, S4)
# union of sets:
C = union(S1, S2)
# intersection of sets:
D = intersect(S1, S3)
# set difference:
E = setdiff(S3, S1)

```

```

# subset check:
issubset(S1, S4)
# add an element:
push!(S4, 99)
# remove the last element:
pop!(S4)

```

4.1.4 Массивы

```

# empty array with abstract type:
empty_array_1 = []
# empty arrays with concrete types:
empty_array_2 = (Int64)[]
empty_array_3 = (Float64)[]
# column vector:
a = [1, 2, 3]
# row vector:
b = [1 2 3]
# matrices:
A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]
B = [[1 2 3]; [4 5 6]; [7 8 9]]
# 1x8 array of random values in [0, 1):
c = rand(1,8)
# 2x3 array of random values in [0, 1):
C = rand(2,3);
# 3D array:
D = rand(4, 3, 2)
# array of square roots from 1 to 10:

```

```

roots = [sqrt(i) for i in 1:10]
# array of 3*i^2 for odd i from 1 to 9:
ar_1 = [3*i^2 for i in 1:2:9]
# array of squares not divisible by 5 or 4:
ar_2 = [i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]

```

4.2 Задание №1

```

A = Set([0,3,4,9])
B = Set([1,3,4,7])
C = Set([0,1,2,4,7,8,9])
P = intersect(union(intersect(union(intersect(union(intersect(A,B),A),B),C),A),B),C)

```

Результат: Set([0, 4, 7, 9, 1]).

4.3 Задание №2. Операции над множествами элементов разных типов

```

A = Set([1, 2, 3, 4, 5])
B = Set([4, 5, 6, 7, 8])

println("A = ", A)
println("B = ", B)
println("A cup B = ", union(A, B))
println("A cap B = ", intersect(A, B))
println("A setminus B = ", setdiff(A, B))
println("B setminus A = ", setdiff(B, A))

```

```
println("A triangle B = ", symdiff(A, B))
println("3 in A? ", 3 in A)
println("10 in A? ", 10 in A)
println("Set([1,2]) subseteq A? ", issubset(Set([1,2]), A))
println("|A| = ", length(A))
```

```
A = Set([5, 4, 2, 3, 1])
B = Set([5, 4, 6, 7, 8])
A cup B = Set([5, 4, 6, 7, 2, 8, 3, 1])
A cap B = Set([5, 4])
A setminus B = Set([2, 3, 1])
B setminus A = Set([6, 7, 8])
A triangle B = Set([6, 7, 2, 8, 3, 1])
3 in A? true
10 in A? false
Set([1,2]) subseteq A? true
|A| = 5
```

```
langs1 = Set(["Julia", "Python", "R", "C++"])
langs2 = Set(["Python", "Rust", "Go", "C++"])
println("Union: ", union(langs1, langs2))
println("Intersection: ", intersect(langs1, langs2))
println("Difference: ", setdiff(langs1, langs2))
println("Julia in langs1? ", "Julia" in langs1)
```

```
Union: Set(["Go", "Rust", "Julia", "Python", "C++", "R"])
Intersection: Set(["C++", "Python"])
Difference: Set(["Julia", "R"])
Julia in langs1? true
```

```
A = Set{Any}([1, 2, 3, "4", 5.0])
B = Set{Any}([3, "4", 6, 5.0, 'x'])
println("Common elements: ", intersect(A, B))
println("Only in A: ", setdiff(A, B))
println("Only in B: ", setdiff(B, A))
```

Common elements: Set{Any[5.0, "4", 3]}

Only in A: Set{Any[2, 1]}

Only in B: Set{Any[6, 'x']}

```
P = Set([(1, 2), (3, 4), (1, 2)])
println("P = ", P)
push!(P, (5, 6))
println("After push!: ", P)
println("(3,4) in P? ", (3,4) in P)
println("(7,8) in P? ", (7,8) in P)
```

P = Set([(1, 2), (3, 4)])

After push!: Set([(1, 2), (3, 4), (5, 6)])

(3,4) in P? true

(7,8) in P? false

4.4 Задание №3

4.4.1 3.1–3.13. Построение массивов и векторов

```
using Random
Random.seed!(42)
```

```

# 3.1) array (1, 2, 3, ..., N-1, N), N > 20
println("\n--- 3.1 ---")
N = 25
a31 = collect(1:N)
println("N = $N")
println("a31 = ", a31)

# 3.2) array (N, N-1, ..., 2, 1), N > 20
println("\n--- 3.2 ---")
a32 = collect(N:-1:1)
println("a32 = ", a32)

# 3.3) array (1, 2, ..., N-1, N, N-1, ..., 2, 1), N > 20
println("\n--- 3.3 ---")
a33 = vcat(1:N, (N-1):-1:1)
println("a33 = ", a33)

# 3.4) array tmp of the form (4, 6, 3)
println("\n--- 3.4 ---")
tmp = [4, 6, 3]
println("tmp = ", tmp)

# 3.5) first element of tmp repeated 10 times
println("\n--- 3.5 ---")
a35 = fill(tmp[1], 10)
println("a35 = ", a35)

# 3.6) all elements of tmp repeated 10 times

```

```

println("\n--- 3.6 ---")
a36 = repeat(tmp, 10)
println("a36 = ", a36)

# 3.7) 1st element – 11 times, 2nd – 10 times, 3rd – 10 times
println("\n--- 3.7 ---")
a37 = vcat(fill(tmp[1], 11), fill(tmp[2], 10), fill(tmp[3], 10))
println("a37 = ", a37)

# 3.8) 1st element – 10 times, 2nd – 20 times, 3rd – 30 times
println("\n--- 3.8 ---")
a38 = vcat(fill(tmp[1], 10), fill(tmp[2], 20), fill(tmp[3], 30))
println("a38 = ", a38)

# 3.9) array of 2^tmp[i]; 2^tmp[3] appears 4 times;
#       count occurrences of digit 6
println("\n--- 3.9 ---")
base = [2^tmp[1], 2^tmp[2], 2^tmp[3]]
a39 = vcat(base, fill(2^tmp[3], 4))
println("a39 = ", a39)
cnt6 = 0
for v in a39
    cnt6 += count(==( '6' ), string(v))
end
println("Digit 6 occurs: ", cnt6)

# 3.10) y = e^x * cos(x), x = 3, 3.1, ..., 6; mean of y
println("\n--- 3.10 ---")

```

```

xs = 3.0:0.1:6.0
ys = [exp(x) * cos(x) for x in xs]
println("xs = ", collect(xs))
println("ys = ", ys)
println("Mean y = ", sum(ys) / length(ys))

# 3.11) vector (x^i, y^j), x=0.1, i=3,6,...,36; y=0.2, j=1,4,...,34
println("\n--- 3.11 ---")
x = 0.1
y = 0.2
a311 = [(x^i, y^j) for i in 3:3:36 for j in 1:3:34]
println("a311 = ", a311)

# 3.12) vector 2^i / i, i = 1..M, M = 25
println("\n--- 3.12 ---")
M = 25
a312 = [2^i / i for i in 1:M]
println("a312 = ", a312)

# 3.13) vector ("fn1", "fn2", ..., "fnN"), N = 30
println("\n--- 3.13 ---")
N13 = 30
a313 = ["fn$i" for i in 1:N13]
println("a313 = ", a313)

```

4.4.2 3.14. Работа с векторами x и y длины n = 250

```
println("\n--- 3.14 ---")
n = 250
x = rand(0:999, n)
y = rand(0:999, n)
println("x = ", x)
println("y = ", y)

# vector (y2 - x1, ..., yn - x_{n-1})
println("\n-- 3.14 (y2-x1, ..., yn-x_{n-1}) --")
v1 = [y[i+1] - x[i] for i in 1:n-1]
println("v1 = ", v1)

# vector (x1+2x2-x3, ..., x_{n-2}+2x_{n-1}-x_n)
println("\n-- 3.14 (x1+2x2-x3, ..., x_{n-2}+2x_{n-1}-x_n) --")
v2 = [x[i] + 2x[i+1] - x[i+2] for i in 1:n-2]
println("v2 = ", v2)

# vector (sin(y_i)/cos(x_{i+1}))
println("\n-- 3.14 (sin(y_i)/cos(x_{i+1})) --")
v3 = [sin(y[i]) / cos(x[i+1]) for i in 1:n-1]
println("v3 = ", v3)

# sum from i=1 to n-1 of exp(-x_{i+1}) / (x_i + 10)
println("\n-- 3.14 sum --")
s = sum(exp(-x[i+1]) / (x[i] + 10) for i in 1:n-1)
println("Sum = ", s)
```

4.4.3 Пункты со стр. 18

```
# elements of y greater than 600 and their indices
println("\n-- y > 600 --")
mask = y .> 600
println("Elements y > 600: ", y[mask])
println("Indices: ", findall(mask))

# x values corresponding to y > 600
println("\n-- x corresponding to y > 600 --")
println(x[mask])

# vector ( $|x_i - x_{\text{bar}}|^{(1/2)}$ )
println("\n-- ( $|x_i - x_{\text{bar}}|^{(1/2)}$ ) --")
x_bar = sum(x) / length(x)
println([sqrt(abs(xi - x_bar)) for xi in x])

# count of y within 200 from maximum
println("\n-- y within 200 from maximum --")
ymax = maximum(y)
println("Count: ", count(yi -> abs(yi - ymax) <= 200, y))

# number of even/odd elements of x
println("\n-- even/odd x --")
println("Even: ", count(iseven, x))
println("Odd: ", count(isodd, x))

# number of x divisible by 7
println("\n-- x divisible by 7 --")
```

```
println("Count: ", count(xi -> xi % 7 == 0, x))

# sort x by y in ascending order
println("\n-- x sorted by y --")
println(x[sortperm(y)])

# top-10 largest x
println("\n-- top-10 x --")
println(sort(x, rev=true)[1:10])

# unique elements of x
println("\n-- unique x --")
println(unique(x))
```

4.5 Задание №4. Массив squares

```
# array of squares of all even numbers from 1 to 100
squares = [i^2 for i in 1:100 if iseven(i)]
println("squares = ", squares)
println("Number of elements: ", length(squares))
```

squares = [4, 16, 36, 64, 100, 144, 196, 256, 324, 400, 484, 576, 676, 784, 884, 984, 1084, 1184, 1284, 1384, 1484, 1584, 1684, 1784, 1884, 1984, 2084, 2184, 2284, 2384, 2484, 2584, 2684, 2784, 2884, 2984, 3084, 3184, 3284, 3384, 3484, 3584, 3684, 3784, 3884, 3984, 4084, 4184, 4284, 4384, 4484, 4584, 4684, 4784, 4884, 4984, 5084, 5184, 5284, 5384, 5484, 5584, 5684, 5784, 5884, 5984, 6084, 6184, 6284, 6384, 6484, 6584, 6684, 6784, 6884, 6984, 7084, 7184, 7284, 7384, 7484, 7584, 7684, 7784, 7884, 7984, 8084, 8184, 8284, 8384, 8484, 8584, 8684, 8784, 8884, 8984, 9084, 9184, 9284, 9384, 9484, 9584, 9684, 9784, 9884, 9984]

Number of elements: 50

4.6 Задание №5. Пакет Primes

```

using Pkg
Pkg.add("Primes")
using Primes

# generate an array of the first 168 primes
primes_arr = primes(1000)[1:168]
println("First 168 primes: ", primes_arr)

# find the 89-th smallest prime
p89 = primes_arr[89]
println("89-th smallest prime: ", p89)

# slice from 89 to 99 inclusive
slice_89_99 = primes_arr[89:99]
println("Slice primes_arr[89:99] = ", slice_89_99)

```

First 168 primes: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167]

89-th smallest prime: 461

Slice primes_arr[89:99] = [461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563, 569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643, 647, 653, 659, 661, 673, 683, 691, 701, 709, 713, 727, 733, 739, 743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 823, 827, 833, 839, 853, 857, 863, 877, 883, 893, 907, 911, 919, 929, 937, 941, 947, 953, 967, 971, 977, 983, 991, 997]

4.7 Задание №6. Вычисление выражений

```

# 6.1) sum from i=10 to 100 of (i^3 + 4i^2)
s61 = sum(i^3 + 4i^2 for i in 10:100)
println("6.1) = ", s61)

# 6.2) sum from i=1 to M of (2i/i + 3i/i^2), M = 25
M = 25

```

```

s62 = sum((2i)/i + (3i)/(i^2) for i in 1:M)
println("6.2) = ", s62)

# 6.3)  $1 + 2/3 + (2/3 * 4/5) + (2/3 * 4/5 * 6/7) + \dots + (2/3 * 4/5 * \dots * 20/21)$ 
function prod_chain(k)
    p = 1.0
    for j in 1:k
        p *= (2j) / (2j + 1)
    end
    return p
end

s63 = 1.0 + sum(prod_chain(k) for k in 1:19)
println("6.3) = ", s63)

```

6.1) = 26852735

6.2) = 61.44787453326052

6.3) = 6.976346137897619

5 Результаты

В ходе работы получены следующие результаты.

Задание №1. Результат: $\text{Set}([0, 4, 7, 9, 1])$.

Задание №2. Продемонстрированы все базовые операции над множествами, в том числе над множествами элементов разных типов ($\text{Set}\{\text{Any}\}$):

- объединение $A \cup B$;
- пересечение $A \cap B$;
- разность $A \setminus B$ и $B \setminus A$;
- симметрическая разность $A \triangle B$;
- проверка принадлежности $x \in A$;
- проверка подмножества $A \subseteq B$;
- мощность множества $|A|$.

Задание №3. Построены массивы 3.1–3.13. Сформированы векторы по заданию 3.14:

- $(y_2 - x_1, y_3 - x_2, \dots, y_n - x_{n-1})$ – длина $n - 1 = 249$;
- $(x_1 + 2x_2 - x_3, \dots, x_{n-2} + 2x_{n-1} - x_n)$ – длина $n - 2 = 248$;
- $\left(\frac{\sin(y_1)}{\cos(x_2)}, \dots, \frac{\sin(y_{n-1})}{\cos(x_n)}\right)$ – длина $n - 1 = 249$;
- сумма $\sum_{i=1}^{n-1} \frac{e^{-x_{i+1}}}{x_i + 10} \approx 1.597 \cdot 10^{-4}$.

Для векторов x и y длины $n = 250$ получены:

- количество элементов $y > 600$ и их индексы;

- значения x , соответствующие $y > 600$;
- вектор $(|x_1 - \bar{x}|^{1/2}, \dots, |x_n - \bar{x}|^{1/2})$, где $\bar{x}$ — среднее x ;
- количество элементов y в пределах 200 от максимума;
- количество чётных и нечётных элементов x ;
- количество элементов x , кратных 7;
- x , отсортированный по возрастанию y ;
- top-10 наибольших элементов x ;
- уникальные элементы x .

Задание №4. Массив `squares` содержит 50 элементов — квадратов чётных чисел от 1 до 100: 4, 16, 36, ..., 10000.

Задание №5. Первые 168 простых чисел получены. 89-е простое число равно 461. Срез с 89-го по 99-й элемент: 461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523.

Задание №6. Получены значения:

- 6.1) $\sum_{i=10}^{100} (i^3 + 4i^2) = 26\,852\,735$;
- 6.2) $\sum_{i=1}^{25} \left(\frac{2i}{i} + \frac{3i}{i^2} \right) \approx 61.4479$;
- 6.3) $1 + \frac{2}{3} + \left(\frac{2}{3} \cdot \frac{4}{5} \right) + \dots + \left(\frac{2}{3} \cdot \frac{4}{5} \dots \frac{38}{39} \right) \approx 6.9763$.

6 Заключение

- **Цель работы достигнута:** изучены структуры данных Julia (кортежи, словари, множества, массивы) и операции над ними.
- **Получены конкретные результаты** по всем шести заданиям.
- **Рекомендации:** при работе с разнотипными множествами использовать `Set{Any}`; для построения массивов удобно применять генераторы и функции `fill`, `repeat`, `vcat`.

Список литературы

1. *Balbaert I.* Getting Started with Julia Programming. — Birmingham : Packt Publishing, 2015. — 214 с.
2. Julia: A Fast Dynamic Language for Technical Computing / J. Bezanson [и др.]. — Cambridge, MA : MIT Press, 2021. — 400 с.
3. Primes.jl — Julia package for prime number utilities. — 2024. — Accessed: 2026-09-14. <https://github.com/JuliaMath/Primes.jl>.
4. The Julia Language Documentation. — 2024. — Accessed: 2026-09-14. <https://docs.julialang.org/>.

А Приложение А. Полный листинг программы

Полный листинг приведён в разделе «Выполнение лабораторной работы» (блоки кода по каждому заданию).

В Приложение Б. Исходный ноутбук

Исходный Jupyter-ноутбук с выполненными заданиями прилагается к отчёту отдельным файлом `.ipynb`.